

Compiled PDF
Project: .

Project Structure: .

=====

```
|-- images/
|   |__ all_to_pdf_logo.png
|-- test/
|   |-- fixtures/
|   |   |__ sample-project/
|   |       |-- data/
|   |           |__ config.json
|   |       |-- src/
|   |           |-- index.js
|   |           |__ utils.py
|   |       |-- .gitignore
|   |       |-- PDF Compiled Project.pdf
|   |       |__ README.md
|   |__ extension.test.js
|-- .eslintrc.json
|-- .vscode-test.mjs
|-- .vscodeignore
|-- CHANGELOG.md
|-- extension.js
|-- jsconfig.json
|-- package.json
|__ pdfGenerator.js
```

(16 files total)

File: .eslintrc.json

```
{
  "env": {
    "browser": false,
    "commonjs": true,
    "es6": true,
    "node": true,
    "mocha": true
  },
  "parserOptions": {
    "ecmaVersion": 2018,
    "ecmaFeatures": {
      "jsx": true
    },
    "sourceType": "module"
  },
  "rules": {
    "no-const-assign": "warn",
    "no-this-before-super": "warn",
    "no-undef": "warn",
    "no-unreachable": "warn",
    "no-unused-vars": "warn",
    "constructor-super": "warn",
    "valid-typeof": "warn"
  }
}
```

Change Log

[0.0.2] – 2026-06-28

Added

- Line numbers on all code file contents
- Project directory tree as the second page
- Table of Contents page with page numbers for every file
- Visual separator lines between file headers and content
- File filtering: skips `node\_modules`, `.git`, `dist`, `build`, and more
- `.gitignore` rule support (opt-out via `allToPdf.useGitIgnore`)
- Three configurable settings: `useGitIgnore`, `exclude`, `maxFileSizeKb`
- Progress notification during PDF generation

Fixed

- Multi-line content now renders correctly (pdf-lib `drawText` doesn't handle `\n`)
- Courier monospace font for code readability
- Binary files no longer crash the generation process
- Page-break logic: clean breaks between lines instead of mid-word

Changed

- Extracted core logic into `pdfGenerator.js` for testability
- Mocha unit test suite with 16 tests

[0.0.1] – Initial release

- Basic PDF generation from project folders
- Support for code, data, image, and text files
- Header page and footer with GitHub link
- Context menu integration

File: extension.js

```
1 | const vscode = require('vscode');
2 | const path = require('path');
3 | const { createPDF } = require('./pdfGenerator');
4 |
5 | function getSettings() {
6 |     const config = vscode.workspace.getConfiguration('allToPdf');
7 |     return {
8 |         useGitIgnore: config.get('useGitIgnore', true),
9 |         skipDirs: config.get('exclude', []),
10 |         maxFileSizeKb: config.get('maxFileSizeKb', 1024)
11 |     };
12 | }
13 |
14 | async function createPDFWithProgress(folderPath) {
15 |     const settings = getSettings();
16 |     await vscode.window.withProgress({
17 |         location: vscode.ProgressLocation.Notification,
18 |         title: 'Generating PDF...',
19 |         cancellable: false
20 |     }, async () => {
21 |         const { pdfBytes, folderName } = await createPDF(folderPath, settings);
22 |         const outputPath = path.join(folderPath, 'PDF Compiled Project.pdf');
23 |         require('fs-extra').writeFile(outputPath, pdfBytes);
24 |         vscode.window.showInformationMessage('PDF successfully generated by All-to-PDF Extension!');
25 |     });
26 | }
27 |
28 | function activate(context) {
29 |     console.log('Congratulations, You can now call the extension by right-clicking the desired project folder and selecting the Compile Folder to PDF option.');
```

option.');

```
30 |
31 |     let disposable = vscode.commands.registerCommand('extension.convertProjectToPDF', async (folder) => {
32 |         if (folder && folder.fsPath) {
33 |             await createPDFWithProgress(folder.fsPath);
34 |         }
35 |     });
36 |
37 |     let context_disposable = vscode.commands.registerCommand('extension.convertProjectToPDFContext', async (folder) => {
38 |         if (folder && folder.fsPath) {
39 |             await createPDFWithProgress(folder.fsPath);
40 |         }
41 |     });
42 |
43 |     context.subscriptions.push(disposable, context_disposable);
44 | }
45 |
46 | function deactivate() {}
47 |
```

File: extension.js

```
48 | module.exports = { activate, deactivate };  
49 |
```



File: jsconfig.json

```
{
  "compilerOptions": {
    "module": "Node16",
    "target": "ES2022",
    "checkJs": false, /* Typecheck .js files. */
    "lib": [
      "ES2022"
    ],
  },
  "exclude": [
    "node_modules"
  ]
}
```


File: package.json

```
{
  "name": "all-to-pdf",
  "displayName": "Project to PDF - AI Facilitator",
  "description": "Compile your entire project folder into a single PDF with line numbers, table of contents, directory tree, .gitignore support, and AI-friendly formatting.",
  "version": "0.0.1",
  "engines": {
    "vscode": "^1.88.0"
  },
  "icon": "images/all_to_pdf_logo.png",
  "galleryBanner": {
    "color": "#ffffff",
    "theme": "light"
  },
  "categories": [
    "Other"
  ],
  "activationEvents": [],
  "main": "./extension.js",
  "contributes": {
    "commands": [
      {
        "command": "extension.convertProjectToPDF",
        "title": "Compile Folder to PDF"
      },
      {
        "command": "extension.convertProjectToPDFContext",
        "title": "Compile Folder to PDF"
      }
    ],
    "menus": {
      "explorer/context": [
        {
          "command": "extension.convertProjectToPDFContext",
          "group": "yourGroup@1",
          "when": "explorerResourceIsFolder"
        }
      ]
    },
    "configuration": {
      "title": "All to PDF",
      "properties": {
        "allToPdf.useGitIgnore": {
          "type": "boolean",
          "default": true,
          "description": "Respect .gitignore rules when collecting files"
        },
        "allToPdf.exclude": {
```

File: package.json

```
"type": "array",
"items": {
  "type": "string"
},
"default": [],
"description": "Additional directory or file names to exclude (e.g. 'my-secrets')"
},
"allToPdf.maxFileSizeKb": {
  "type": "number",
  "default": 1024,
  "description": "Maximum file size in KB to include in the PDF (0 for no limit)"
}
},
"scripts": {
  "lint": "eslint .",
  "pretest": "npm run lint",
  "test": "node ./node_modules/mocha/bin/mocha ./test"
},
"devDependencies": {
  "@types/mocha": "^10.0.6",
  "@types/node": "^18.19.31",
  "@types/vscode": "^1.88.0",
  "@vscode/test-cli": "^0.0.8",
  "@vscode/test-electron": "^2.3.9",
  "eslint": "^8.57.0",
  "mocha": "^10.4.0",
  "typescript": "^5.4.5",
  "vscode": "^1.1.37"
},
"dependencies": {
  "@pdf-lib/fontkit": "^1.1.1",
  "fs-extra": "^11.2.0",
  "ignore": "^7.0.5",
  "jszip": "^3.10.1",
  "pdf-lib": "^1.17.1"
},
"repository": {
  "type": "git",
  "url": "https://github.com/HerreraCarlos81/all_to_pdf"
}
}
```

File: pdfGenerator.js

```
1 | const fs = require('fs-extra');
2 | const pdf = require('pdf-lib');
3 | const path = require('path');
4 | const ignore = require('ignore');
5 |
6 | const PAGE_WIDTH = 841.89;
7 | const PAGE_HEIGHT = 595.28;
8 | const MARGIN_LEFT = 50;
9 | const MARGIN_BOTTOM = 50;
10 | const MARGIN_TOP = 60;
11 |
12 | const DEFAULT_SKIP_DIRS = new Set([
13 |   'node_modules', '.git', 'dist', 'build', '.next', '.nuxt',
14 |   'target', 'out', '.output', '.vercel', '.cache',
15 |   '__pycache__', '.venv', 'venv', 'env', '.env',
16 |   'vendor', '.bundle', 'coverage', '.nyc_output'
17 | ]);
18 |
19 | const DEFAULT_SKIP_FILES = new Set([
20 |   'package-lock.json', 'yarn.lock', 'pnpm-lock.yaml',
21 |   '.DS_Store', 'Thumbs.db'
22 | ]);
23 |
24 | async function loadGitIgnoreRules(folderPath) {
25 |   const ig = ignore();
26 |   const gitignorePath = path.join(folderPath, '.gitignore');
27 |   try {
28 |     const content = await fs.readFile(gitignorePath, 'utf8');
29 |     ig.add(content);
30 |   } catch {
31 |     // No .gitignore found
32 |   }
33 |   return ig;
34 | }
35 |
36 | function shouldSkip(entryPath, entryName, isDir, options) {
37 |   const { skipDirs, skipFiles, gitIgnoreFilter, rootPath } = options;
38 |
39 |   if (isDir) {
40 |     if (skipDirs.has(entryName)) return true;
41 |   } else {
42 |     if (skipFiles.has(entryName)) return true;
43 |   }
44 |
45 |   if (gitIgnoreFilter) {
46 |     const relativePath = path.relative(rootPath, entryPath).replace(/\\/g, '/');
47 |     if (gitIgnoreFilter.ignores(relativePath)) return true;
48 |     if (isDir && gitIgnoreFilter.ignores(`${relativePath}/`)) return true;
```

File: pdfGenerator.js

```
49 |     }
50 |
51 |     return false;
52 | }
53 |
54 | async function collectFiles(folderPath, options = {}) {
55 |     const files = [];
56 |     const entries = await fs.readdir(folderPath, { withFileTypes: true });
57 |
58 |     for (const entry of entries) {
59 |         const fullPath = path.join(folderPath, entry.name);
60 |         if (entry.isDirectory()) {
61 |             if (shouldSkip(fullPath, entry.name, true, options)) continue;
62 |             const subFiles = await collectFiles(fullPath, options);
63 |             files.push(...subFiles);
64 |         } else {
65 |             if (shouldSkip(fullPath, entry.name, false, options)) continue;
66 |             files.push(fullPath);
67 |         }
68 |     }
69 |
70 |     return files;
71 | }
72 |
73 | function generateProjectTree(files, rootPath) {
74 |     const root = {};
75 |     for (const file of files) {
76 |         const relative = path.relative(rootPath, file);
77 |         const parts = relative.split(path.sep);
78 |         let node = root;
79 |         for (let i = 0; i < parts.length; i++) {
80 |             const part = parts[i];
81 |             if (i === parts.length - 1) {
82 |                 node[part] = null;
83 |             } else {
84 |                 if (!node[part] || node[part] === null) node[part] = {};
85 |                 node = node[part];
86 |             }
87 |         }
88 |     }
89 |     return renderTreeLines(root, '');
90 | }
91 |
92 | function renderTreeLines(node, prefix) {
93 |     const entries = Object.entries(node).sort((a, b) => {
94 |         const aIsDir = a[1] !== null;
95 |         const bIsDir = b[1] !== null;
96 |         if (aIsDir && !bIsDir) return -1;
```

```
97 |         if (!aIsDir && bIsDir) return 1;
98 |         return a[0].localeCompare(b[0]);
99 |     });
100 |     let result = '';
101 |     for (let i = 0; i < entries.length; i++) {
102 |         const [key, value] = entries[i];
103 |         const isLast = i === entries.length - 1;
104 |         const connector = isLast ? '|__ ' : '|-- ';
105 |         const suffix = value !== null ? '/' : '';
106 |         result += prefix + connector + key + suffix + '\n';
107 |         if (value !== null) {
108 |             const childPrefix = prefix + (isLast ? ' ' : '| ');
109 |             result += renderTreeLines(value, childPrefix);
110 |         }
111 |     }
112 |     return result;
113 | }
114 |
115 | function addLineNumbers(content, startLine) {
116 |     const lines = content.split('\n');
117 |     const digits = String(lines.length + startLine - 1).length;
118 |     return lines.map((line, i) => {
119 |         const num = String(startLine + i).padStart(digits, ' ');
120 |         return `${num} | ${line}`;
121 |     }).join('\n');
122 | }
123 |
124 | async function addContentPages(pdfDoc, filePathText, content, font, fontSize, lineHeight, maxWidth) {
125 |     const lines = content.split('\n');
126 |     let page = pdfDoc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
127 |     let y = PAGE_HEIGHT - MARGIN_TOP;
128 |     let filePathDrawn = false;
129 |
130 |     for (const line of lines) {
131 |         const words = line.split(' ');
132 |         let lineText = '';
133 |
134 |         for (const word of words) {
135 |             const testLine = lineText ? `${lineText} ${word}` : word;
136 |             const testWidth = font.widthOfTextAtSize(testLine, fontSize);
137 |
138 |             if (testWidth > maxWidth && lineText) {
139 |                 if (y - lineHeight < MARGIN_BOTTOM) {
140 |                     page = pdfDoc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
141 |                     y = PAGE_HEIGHT - MARGIN_TOP;
142 |                     filePathDrawn = false;
143 |                 }
144 |                 if (!filePathDrawn) {
```

File: pdfGenerator.js

```
145 |         drawFileHeader(page, filePathText, font);
146 |         filePathDrawn = true;
147 |     }
148 |     page.drawText(lineText, { x: MARGIN_LEFT, y, size: fontSize, font, maxWidth });
149 |     y -= lineHeight;
150 |     lineText = word;
151 | } else {
152 |     lineText = testLine;
153 | }
154 | }
155 |
156 | if (y - lineHeight < MARGIN_BOTTOM) {
157 |     page = pdfDoc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
158 |     y = PAGE_HEIGHT - MARGIN_TOP;
159 |     filePathDrawn = false;
160 | }
161 | if (!filePathDrawn) {
162 |     drawFileHeader(page, filePathText, font);
163 |     filePathDrawn = true;
164 | }
165 | page.drawText(lineText, { x: MARGIN_LEFT, y, size: fontSize, font, maxWidth });
166 | y -= lineHeight;
167 | }
168 | }
169 |
170 | function drawFileHeader(page, filePathText, font) {
171 |     page.drawText(filePathText, { x: MARGIN_LEFT, y: PAGE_HEIGHT - 45, size: 9, font });
172 |     page.drawLine({
173 |         start: { x: MARGIN_LEFT, y: PAGE_HEIGHT - 50 },
174 |         end: { x: PAGE_WIDTH - MARGIN_LEFT, y: PAGE_HEIGHT - 50 },
175 |         thickness: 0.5,
176 |         color: pdf.rgb(0.6, 0.6, 0.6),
177 |     });
178 | }
179 |
180 | function addHeaderPage(pdfDoc, folderName) {
181 |     const page = pdfDoc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
182 |     page.drawText(`Compiled PDF\n\nProject: ${folderName}`, { x: 300, y: PAGE_HEIGHT - 200, size: 36 });
183 | }
184 |
185 | async function addTreePage(pdfDoc, treeText, folderName, fileCount) {
186 |     const page = pdfDoc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
187 |     const font = await pdfDoc.embedFont(pdf.StandardFonts.Courier);
188 |     const lines = treeText.split('\n').filter(l => l.trim());
189 |     let y = PAGE_HEIGHT - MARGIN_TOP + 10;
190 |     page.drawText(`Project Structure: ${folderName}`, { x: MARGIN_LEFT, y, size: 14, font: font });
191 |     y -= 30;
192 |     const separator = '='.repeat(50);
```

File: pdfGenerator.js

```
193 |     page.drawText(separator, { x: MARGIN_LEFT, y, size: 10, font: font });
194 |     y -= 20;
195 |     for (const line of lines) {
196 |         page.drawText(line, { x: MARGIN_LEFT, y, size: 8, font: font });
197 |         y -= 12;
198 |     }
199 |     y -= 10;
200 |     page.drawText(`(${fileCount} files total)`, { x: MARGIN_LEFT, y, size: 10, font: font });
201 | }
202 |
203 | async function addTOCPage(pdfDoc, tocEntries) {
204 |     const page = pdfDoc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
205 |     const font = await pdfDoc.embedFont(pdf.StandardFonts.Courier);
206 |     let y = PAGE_HEIGHT - MARGIN_TOP + 10;
207 |     page.drawText('Table of Contents', { x: MARGIN_LEFT, y, size: 14, font: font });
208 |     y -= 30;
209 |     const separator = '='.repeat(60);
210 |     page.drawText(separator, { x: MARGIN_LEFT, y, size: 10, font: font });
211 |     y -= 20;
212 |     for (const entry of tocEntries) {
213 |         if (y < MARGIN_BOTTOM + 20) break;
214 |         const pageNum = String(entry.page);
215 |         const dots = '.'.repeat(Math.max(1, 60 - entry.path.length - pageNum.length - 2));
216 |         page.drawText(`${entry.path} ${dots} ${pageNum}`, { x: MARGIN_LEFT, y, size: 8, font: font });
217 |         y -= 12;
218 |     }
219 | }
220 |
221 | async function addImageFilePage(pdfDoc, filePathText, file, extension) {
222 |     const imageBytes = await fs.readFile(file);
223 |     const image = extension === '.png' ? await pdfDoc.embedPng(imageBytes) : await pdfDoc.embedJpg(imageBytes);
224 |     const page = pdfDoc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
225 |     const maxDimension = 500;
226 |     let scaledWidth = image.width;
227 |     let scaledHeight = image.height;
228 |     if (image.width > maxDimension || image.height > maxDimension) {
229 |         const scaleFactor = maxDimension / Math.max(image.width, image.height);
230 |         scaledWidth = image.width * scaleFactor;
231 |         scaledHeight = image.height * scaleFactor;
232 |     }
233 |     const x = (page.getWidth() - scaledWidth) / 2;
234 |     const y = ((page.getHeight() - scaledHeight) / 2) - 20;
235 |     page.drawText(filePathText, { x: MARGIN_LEFT, y: page.getHeight() - 50, size: 10 });
236 |     page.drawImage(image, { x, y, width: scaledWidth, height: scaledHeight });
237 | }
238 |
239 | async function addFooterPage(pdfDoc, folderName) {
240 |     const lastPage = pdfDoc.getPages()[pdfDoc.getPageCount() - 1];
```

```
241 |     const currentDate = new Date().toLocaleDateString();
242 |     const footerText = `PDF of project created on ${currentDate} with `;
243 |     const linkText = 'All to PDF - Ai Facilitator';
244 |     const linkUrl = 'https://github.com/HerreraCarlos81/all_to_pdf';
245 |     const extensionText = ' extension for VSCode';
246 |     const footerFontSize = 10;
247 |     const footerWidth = lastPage.getWidth() - 100;
248 |     const footerHeight = footerFontSize * 1.2;
249 |     const footerX = 50;
250 |     const footerY = 50;
251 |
252 |     lastPage.drawRectangle({
253 |         x: footerX,
254 |         y: footerY,
255 |         width: footerWidth,
256 |         height: footerHeight,
257 |         borderWidth: 1,
258 |         borderColor: pdf.rgb(0, 0, 0),
259 |     });
260 |
261 |     const font = await pdfDoc.embedFont(pdf.StandardFonts.Helvetica);
262 |     const textWidth = font.widthOfTextAtSize(footerText, footerFontSize);
263 |     const linkWidth = font.widthOfTextAtSize(linkText, footerFontSize);
264 |
265 |     lastPage.drawText(footerText, {
266 |         x: footerX + 10,
267 |         y: footerY + 2,
268 |         size: footerFontSize,
269 |         font: font,
270 |         color: pdf.rgb(0, 0, 0),
271 |     });
272 |
273 |     const linkX = footerX + 10 + textWidth;
274 |     const linkY = footerY + 2;
275 |     lastPage.drawText(linkText, {
276 |         x: linkX,
277 |         y: linkY,
278 |         size: footerFontSize,
279 |         font: font,
280 |         color: pdf.rgb(0, 0, 1),
281 |         underline: true,
282 |     });
283 |
284 |     const linkAnnotation = pdfDoc.context.obj({
285 |         Type: 'Annot',
286 |         Subtype: 'Link',
287 |         Rect: [linkX, linkY, linkX + linkWidth, linkY + footerFontSize],
288 |         Border: [0, 0, 0],
```



```
289 |         A: {
290 |             Type: 'Action',
291 |             S: 'URI',
292 |             URI: linkUrl,
293 |         },
294 |     });
295 |     lastPage.node.addAnnot(linkAnnotation);
296 |
297 |     lastPage.drawText(extensionText, {
298 |         x: footerX + 10 + textWidth + linkWidth,
299 |         y: footerY + 2,
300 |         size: footerFontSize,
301 |         font: font,
302 |         color: pdf.rgb(0, 0, 0),
303 |     });
304 | }
305 |
306 | function setMetadata(pdfDoc, folderName) {
307 |     pdfDoc.setTitle(folderName);
308 |     pdfDoc.setAuthor('Created with All to PDF - AI Facilitator');
309 |     pdfDoc.setSubject('PDF generated from project files');
310 |     pdfDoc.setKeywords([folderName, 'project', 'compilation', 'PDF']);
311 | }
312 |
313 | async function createPDF(folderPath, options = {}) {
314 |     const skipDirs = new Set([...DEFAULT_SKIP_DIRS, ...(options.skipDirs || [])]);
315 |     const skipFiles = new Set([...DEFAULT_SKIP_FILES, ...(options.skipFiles || [])]);
316 |     const useGitIgnore = options.useGitIgnore !== false;
317 |
318 |     let gitIgnoreFilter = null;
319 |     let rootPath = folderPath;
320 |
321 |     if (useGitIgnore) {
322 |         gitIgnoreFilter = await loadGitIgnoreRules(folderPath);
323 |     }
324 |
325 |     const collectOptions = { skipDirs, skipFiles, gitIgnoreFilter, rootPath };
326 |     const files = await collectFiles(folderPath, collectOptions);
327 |
328 |     const pdfDoc = await pdf.PDFDocument.create();
329 |     const folderName = path.basename(folderPath);
330 |     addHeaderPage(pdfDoc, folderName);
331 |
332 |     const treeText = generateProjectTree(files, folderPath);
333 |     await addTreePage(pdfDoc, treeText, folderName, files.length);
334 |
335 |     const font = await pdfDoc.embedFont(pdf.StandardFonts.Courier);
336 |     const tocEntries = [];
```

```
337 |
338 |   for (const file of files) {
339 |     const extension = path.extname(file).toLowerCase();
340 |     let content;
341 |     try {
342 |       content = await fs.readFile(file, 'utf8');
343 |       content = content.replace(/\r\n/g, '\n').replace(/\r/g, '\n').replace(/\t/g, ' ');
344 |     } catch {
345 |       continue;
346 |     }
347 |     const filePathText = `File: ${file}`;
348 |     const startPage = pdfDoc.getPageCount() + 1;
349 |
350 |     if (['.js', '.ts', '.py', '.java', '.c', '.cpp', '.cs', '.php', '.rb', '.go'].includes(extension)) {
351 |       const numberedContent = addLineNumbers(content, 1);
352 |       await addContentPages(pdfDoc, filePathText, numberedContent, font, 8, 10, 740);
353 |     } else if (['.json', '.csv', '.xml', '.yaml', '.yml'].includes(extension)) {
354 |       try {
355 |         const data = extension === '.json' ? JSON.parse(content) : content;
356 |         const formattedContent = JSON.stringify(data, null, 2);
357 |         await addContentPages(pdfDoc, filePathText, formattedContent, font, 8, 10, 740);
358 |       } catch {
359 |         await addContentPages(pdfDoc, filePathText, content, font, 8, 10, 740);
360 |       }
361 |     } else if (['.jpg', '.jpeg', '.png', '.gif', '.bmp'].includes(extension)) {
362 |       try {
363 |         await addImageFilePage(pdfDoc, filePathText, file, extension);
364 |       } catch {
365 |         continue;
366 |       }
367 |     } else if (['.txt', '.md', '.html', '.css'].includes(extension)) {
368 |       await addContentPages(pdfDoc, filePathText, content, font, 8, 10, 740);
369 |     }
370 |
371 |     const endPage = pdfDoc.getPageCount();
372 |     if (endPage >= startPage) {
373 |       tocEntries.push({ path: path.relative(folderPath, file).replace(/\\/g, '/'), page: startPage });
374 |     }
375 |   }
376 |
377 |   await addTOCPage(pdfDoc, tocEntries);
378 |   await addFooterPage(pdfDoc, folderName);
379 |   setMetadata(pdfDoc, folderName);
380 |   const pdfBytes = await pdfDoc.save();
381 |   return { pdfBytes, folderName };
382 | }
383 |
384 | module.exports = { createPDF, collectFiles, addContentPages, addHeaderPage, addImageFilePage, addFooterPage, setMetadata, loadGitIgnoreRules,
```

File: pdfGenerator.js

```
shouldSkip, DEFAULT_SKIP_DIRS, DEFAULT_SKIP_FILES, generateProjectTree, addLineNumbers, addTreePage, addTOCPage };  
385 |
```

File: test\extension.test.js

```
1 | const assert = require('assert');
2 | const path = require('path');
3 | const fs = require('fs-extra');
4 | const pdf = require('pdf-lib');
5 | const pdfGenerator = require('../pdfGenerator');
6 | const { createPDF, collectFiles, addHeaderPage, setMetadata } = pdfGenerator;
7 |
8 | const FIXTURE_DIR = path.join(__dirname, 'fixtures', 'sample-project');
9 | const OUTPUT_DIR = path.join(__dirname, 'output');
10 |
11 | describe('pdfGenerator', function () {
12 |   before(async function () {
13 |     await fs.ensureDir(OUTPUT_DIR);
14 |   });
15 |
16 |   after(async function () {
17 |     await fs.remove(OUTPUT_DIR);
18 |   });
19 |
20 |   describe('collectFiles', function () {
21 |     const defaultCollectOpts = {
22 |       skipDirs: pdfGenerator.DEFAULT_SKIP_DIRS,
23 |       skipFiles: pdfGenerator.DEFAULT_SKIP_FILES,
24 |       gitIgnoreFilter: null,
25 |       rootPath: FIXTURE_DIR
26 |     };
27 |
28 |     it('should collect all files recursively from a directory', async function () {
29 |       const files = await collectFiles(FIXTURE_DIR, defaultCollectOpts);
30 |       const relativeFiles = files.map(f => path.relative(FIXTURE_DIR, f)).sort();
31 |       assert.ok(relativeFiles.includes(path.join('src', 'index.js')));
32 |       assert.ok(relativeFiles.includes(path.join('src', 'utils.py')));
33 |       assert.ok(relativeFiles.includes(path.join('data', 'config.json')));
34 |       assert.ok(relativeFiles.includes('README.md'));
35 |     });
36 |
37 |     it('should skip default directories like node_modules', async function () {
38 |       const nmDir = path.join(FIXTURE_DIR, 'node_modules');
39 |       const nmFile = path.join(nmDir, 'lodash', 'index.js');
40 |       await fs.ensureDir(path.dirname(nmFile));
41 |       await fs.writeFile(nmFile, 'module.exports = {};');
42 |       try {
43 |         const files = await collectFiles(FIXTURE_DIR, defaultCollectOpts);
44 |         const relative = files.map(f => path.relative(FIXTURE_DIR, f));
45 |         assert.ok(!relative.includes('node_modules'));
46 |         assert.ok(!relative.some(f => f.startsWith('node_modules')));
47 |       } finally {
48 |         await fs.remove(nmDir);
```

```
49 |     }
50 |   });
51 |
52 |   it('should return an empty array for an empty directory', async function () {
53 |     const emptyDir = path.join(OUTPUT_DIR, 'empty');
54 |     await fs.ensureDir(emptyDir);
55 |     const files = await collectFiles(emptyDir, defaultCollectOpts);
56 |     assert.strictEqual(files.length, 0);
57 |   });
58 | });
59 |
60 | describe('createPDF', function () {
61 |   it('should generate a valid PDF from the sample project', async function () {
62 |     const result = await createPDF(FIXTURE_DIR);
63 |     assert.ok(result.pdfBytes);
64 |     assert.ok(result.pdfBytes instanceof Uint8Array);
65 |     assert.ok(result.pdfBytes.length > 0);
66 |     assert.strictEqual(result.folderName, 'sample-project');
67 |   });
68 |
69 |   it('should generate a PDF with all expected pages', async function () {
70 |     const { pdfBytes } = await createPDF(FIXTURE_DIR);
71 |     const pdfDoc = await pdf.PDFDocument.load(pdfBytes);
72 |     assert.ok(pdfDoc.getPageCount() >= 4,
73 |       `Expected at least 4 pages (header + tree + content + TOC), got ${pdfDoc.getPageCount()}`);
74 |   });
75 |
76 |   it('should set PDF metadata', async function () {
77 |     const { pdfBytes, folderName } = await createPDF(FIXTURE_DIR);
78 |     const pdfDoc = await pdf.PDFDocument.load(pdfBytes);
79 |     assert.strictEqual(pdfDoc.getTitle(), folderName);
80 |     assert.strictEqual(pdfDoc.getAuthor(), 'Created with All to PDF - AI Facilitator');
81 |   });
82 | });
83 |
84 | describe('addHeaderPage', function () {
85 |   it('should add a header page to the PDF', async function () {
86 |     const pdfDoc = await pdf.PDFDocument.create();
87 |     addHeaderPage(pdfDoc, 'test-project');
88 |     assert.strictEqual(pdfDoc.getPageCount(), 1);
89 |   });
90 | });
91 |
92 | describe('setMetadata', function () {
93 |   it('should set title, author, subject, and keywords', async function () {
94 |     const pdfDoc = await pdf.PDFDocument.create();
95 |     setMetadata(pdfDoc, 'my-project');
96 |     assert.strictEqual(pdfDoc.getTitle(), 'my-project');
```

File: test\extension.test.js

```
97 |         assert.strictEqual(pdfDoc.getAuthor(), 'Created with All to PDF - AI Facilitator');
98 |         assert.strictEqual(pdfDoc.getSubject(), 'PDF generated from project files');
99 |     });
100 | });
101 |
102 | describe('File filtering', function () {
103 |     it('should respect .gitignore rules when gitIgnoreFilter is provided', async function () {
104 |         const filteredDir = path.join(OUTPUT_DIR, 'gitignore-test');
105 |         await fs.ensureDir(path.join(filteredDir, 'src'));
106 |         await fs.ensureDir(path.join(filteredDir, 'temp'));
107 |         await fs.writeFile(path.join(filteredDir, 'src', 'index.js'), 'ok');
108 |         await fs.writeFile(path.join(filteredDir, 'temp', 'cache.txt'), 'ignored');
109 |         await fs.writeFile(path.join(filteredDir, '.gitignore'), 'temp/');
110 |
111 |         const gitIgnoreFilter = await pdfGenerator.loadGitIgnoreRules(filteredDir);
112 |         const opts = {
113 |             skipDirs: new Set(),
114 |             skipFiles: new Set(),
115 |             gitIgnoreFilter,
116 |             rootPath: filteredDir
117 |         };
118 |         const files = await collectFiles(filteredDir, opts);
119 |         const relative = files.map(f => path.relative(filteredDir, f));
120 |         assert.ok(relative.includes(path.join('src', 'index.js')));
121 |         assert.ok(!relative.some(f => f.startsWith('temp')));
122 |         await fs.remove(filteredDir);
123 |     });
124 | });
125 |
126 | describe('addLineNumbers', function () {
127 |     it('should add line numbers to content', function () {
128 |         const result = pdfGenerator.addLineNumbers('a\nb\nc', 1);
129 |         assert.strictEqual(result, '1 | a\n2 | b\n3 | c');
130 |     });
131 |
132 |     it('should pad line numbers correctly for multi-digit counts', function () {
133 |         const lines = Array.from({ length: 15 }, (_, i) => `line${i + 1}`);
134 |         const result = pdfGenerator.addLineNumbers(lines.join('\n'), 1);
135 |         const outLines = result.split('\n');
136 |         assert.ok(outLines[0].startsWith(' 1 | '));
137 |         assert.ok(outLines[14].startsWith('15 | '));
138 |     });
139 |
140 |     it('should handle custom start line', function () {
141 |         const result = pdfGenerator.addLineNumbers('hello\nworld', 10);
142 |         assert.strictEqual(result, '10 | hello\n11 | world');
143 |     });
144 | });
```

```
145 |
146 | describe('generateProjectTree', function () {
147 |     it('should generate a tree from a list of files', function () {
148 |         const files = [
149 |             path.join(FIXTURE_DIR, 'src', 'index.js'),
150 |             path.join(FIXTURE_DIR, 'src', 'utils.py'),
151 |             path.join(FIXTURE_DIR, 'README.md')
152 |         ];
153 |         const tree = pdfGenerator.generateProjectTree(files, FIXTURE_DIR);
154 |         assert.ok(tree.includes('index.js'));
155 |         assert.ok(tree.includes('utils.py'));
156 |         assert.ok(tree.includes('README.md'));
157 |         assert.ok(tree.includes('src/'));
158 |     });
159 | });
160 |
161 | describe('TOC generation', function () {
162 |     it('should include correct page numbers in TOC', async function () {
163 |         const { pdfBytes } = await createPDF(FIXTURE_DIR);
164 |         const pdfDoc = await pdf.PDFDocument.load(pdfBytes);
165 |         const pages = pdfDoc.getPageCount();
166 |         assert.ok(pages >= 4);
167 |     });
168 | });
169 |
170 | describe('Bug fixes', function () {
171 |     it('should handle multi-line content without crashing', async function () {
172 |         const multiLine = 'line1\nline2\nline3\nline4\nline5';
173 |         const pdfDoc = await pdf.PDFDocument.create();
174 |         const font = await pdfDoc.embedFont(pdf.StandardFonts.Courier);
175 |         await pdfGenerator.addContentPages(pdfDoc, 'test.js', multiLine, font, 8, 10, 740);
176 |         assert.ok(pdfDoc.getPageCount() >= 1);
177 |     });
178 |
179 |     it('should handle binary/non-utf8 files without crashing', async function () {
180 |         const fixture = path.join(FIXTURE_DIR, 'should-be-skipped.bin');
181 |         await fs.writeFile(fixture, Buffer.from([0x89, 0x50, 0x4E, 0x47])); // PNG header
182 |         try {
183 |             const opts = {
184 |                 skipDirs: pdfGenerator.DEFAULT_SKIP_DIRS,
185 |                 skipFiles: pdfGenerator.DEFAULT_SKIP_FILES,
186 |                 gitIgnoreFilter: null,
187 |                 rootPath: FIXTURE_DIR
188 |             };
189 |             const files = await collectFiles(FIXTURE_DIR, opts);
190 |             assert.ok(files.includes(fixture));
191 |             const result = await createPDF(FIXTURE_DIR);
192 |             assert.ok(result.pdfBytes.length > 0);
```

File: test\extension.test.js

```
193 |         } finally {  
194 |             await fs.remove(fixture);  
195 |         }  
196 |     });  
197 | });  
198 | });  
199 |
```


File: test\fixtures\sample-project\data\config.json

```
{
  "name": "sample-project",
  "version": "1.0.0",
  "description": "A sample project for testing",
  "dependencies": {
    "express": "^4.18.0"
  }
}
```

File: test\fixtures\sample-project\README.md

Sample Project

This is a sample project used for testing the PDF generation.

Features

- Feature 1
- Feature 2

File: test\fixtures\sample-project\src\index.js

```
1 | function greet(name) {  
2 |     return `Hello, ${name}!`;   
3 | }  
4 |  
5 | function fibonacci(n) {  
6 |     if (n <= 1) return n;  
7 |     return fibonacci(n - 1) + fibonacci(n - 2);  
8 | }  
9 |  
10 | module.exports = { greet, fibonacci };  
11 |
```

File: test\fixtures\sample-project\src\utils.py

```
1 | def hello(name):
2 |     return f"Hello, {name}!"
3 |
4 |
5 | def factorial(n):
6 |     if n <= 1:
7 |         return 1
8 |     return n * factorial(n - 1)
9 |
```

Table of Contents

=====

.eslintrc.json	3
CHANGELOG.md	4
extension.js	5
images/all_to_pdf_logo.png	7
jsconfig.json	8
package.json	9
pdfGenerator.js	11
test/extension.test.js	20
test/fixtures/sample-project/data/config.json	25
test/fixtures/sample-project/README.md	26
test/fixtures/sample-project/src/index.js	27
test/fixtures/sample-project/src/utils.py	28